Boosting Metagenomic Taxonomic Classification via Assembly Graph Label Propagation

Pizzo Filippo (2200661), Demo Filippo (2158520)
Department of Information Engineering (DEI)
University of Padua

***Abstract*—Standard read-level taxonomic classification tools like Kraken2 rely on large reference databases that require up to 100 GB of RAM, creating a major computational bottleneck for standard commodity hardware. In this work, we propose a resource-efficient, reproducible workflow orchestrated via Snakemake that combines a memory-constrained classifier (Kraken2 Mini, 10 GB RAM) with assembly graph label propagation (MEGAHIT and GraphBin2) and read back-mapping (Bowtie2). We benchmark our architecture against Kraken2 Standard and Kraken2 Mini on the CAMI II Marine short-read dataset using the AMBER evaluation suite. While Kraken2 Mini suffers a severe drop in taxonomic recovery at the species level (classifying only 35.1% of reads with a sample F_1 score of 0.505), our graph-boosted pipeline effectively rescues unclassified reads, increasing the percentage of binned reads to 72.6% and the sample completeness to 0.673 (F_1 score of 0.779), while maintaining a high sample purity of 0.926. Across all pipeline stages, peak memory usage remains under 14 GB RAM, compared to 103 GB for Kraken2 Standard. Our results demonstrate that assembly graph topology can bridge the performance gap of compressed databases in memory-constrained environments.**

I. Introduction

Metagenomic sequencing generates millions of raw DNA reads directly from environmental samples, requiring fast and accurate taxonomic classification to identify the species and genera present in a microbial community. A prominent state-of-the-art strategy relies on exact k-mer matching against massive reference databases, as implemented by classifiers like Kraken2. To achieve ultra-fast classification speeds, these algorithms store hundreds of millions of k-mer-to-taxon mappings directly in dynamic in-memory hash tables. Consequently, storing complete taxonomic indices demands up to 100 GB of RAM. Offloading this memory footprint to disk via virtual memory swapping is computationally unfeasible; the highly random memory access pattern of k-mer lookups induces severe disk thrashing, reducing throughput by orders of magnitude and creating a critical computational bottleneck for standard commodity hardware and field deployments.

To address this limitation, we investigate whether a resource-efficient workflow, combining a memory-constrained taxonomic classifier with assembly graph label propagation, can achieve classification performance comparable to high-resource baselines. Rather than attempting direct read-level classification with an incomplete database, our strategy shifts the classification target to assembled contigs and leverages their structural connectivity. Our core research hypothesis relies on a two-step framework:

- **Contig Binning:** consisting of the deployment of a lightweight database to assign initial taxonomic labels exclusively to assembled contigs. (Kraken2 Mini, operating within a ~ 10 GB RAM footprint)
- **Label Propagation:** leveraging the assembly graph topology—where nodes represent contigs and edges reflect genomic overlaps and connectivity—to propagate high-confidence labels to unclassified neighboring contigs, which are subsequently back-mapped to the original raw reads.

This process aims to effectively rescue reads that the limited database failed to classify due to missing reference k-mers.

To evaluate the feasibility and trade-offs of this approach, we benchmark our pipeline against two standard paradigms:

1) **Baseline:** direct read classification using Kraken2 with a compressed 8GB database.
2) **Gold Standard:** direct read classification using Kraken2 with the complete, uncompressed ~ 100GB database.

The main contributions of this work are as follows:

- **Automated and Reproducible Workflow:** we design an end-to-end pipeline orchestrated via Snakemake, seamlessly managing complex dependencies between sequence assembly, read alignment, taxonomic binning, and graph-based propagation.
- **Scalable Graph Construction:** we integrate MEGAHIT to generate succinct de Bruijn graphs (SdBG), significantly reducing assembly memory overhead and execution time compared to traditional assemblers like SPAdes, thus keeping the entire preparation phase computationally tractable.
- **Trade-off and Sustainability Evaluation:** we perform a rigorous benchmark on the CAMI II Marine dataset at the species level, analyzing the trade-offs between precision, recall, unclassified read recovery rate, and overall computational resource consumption (RAM, CPU time).

II. Methodology

The proposed framework replaces the standard k-mer lookups, which are heavy in memory, with a hybrid strategy that combines a lightweight read-to-contig classification with an assembly graph topology and an abundance-driven label propagation. In Figure 1, the end-to-end data flow of our approach is represented.

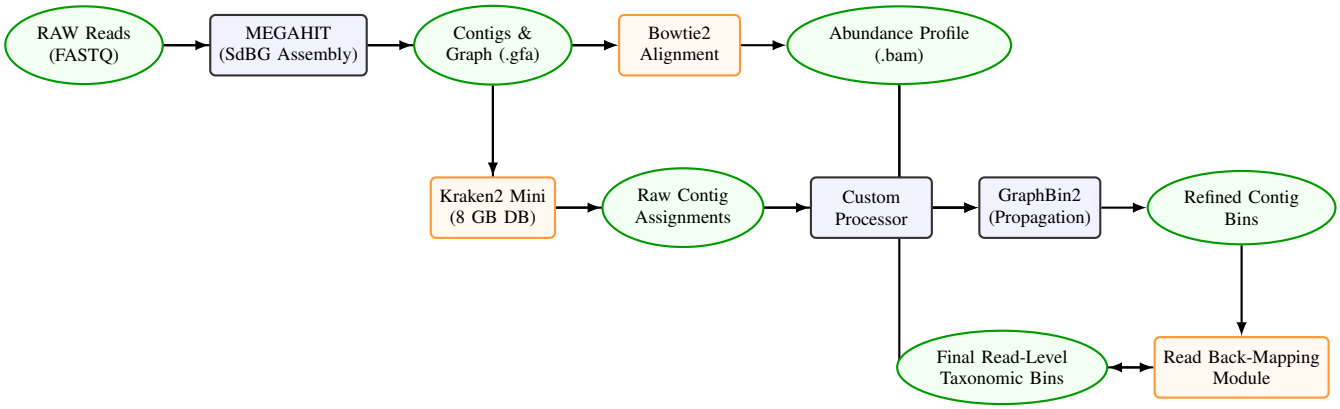


Fig. 1. End-to-end architecture of the graph-boosted taxonomic classification pipeline.

A. Workflow Orchestration and Reproducibility

The execution of our pipeline generates a Directed Acyclic Graph (DAG) with complex dependencies composed of multiple branches. Downstream analysis requires the simultaneous synchronization of three distinct parallel processes:

1) **Graph Topology Stream:** Assembly of raw reads using MEGAHIT to produce both the contig sequences and their structural connectivity graph (`.gfa`).
2) **Taxonomic Seeding Stream:** Contig classification using Kraken2 Mini, followed by some post-processing to output filtered, integer-mapped contig bins.
3) **Abundance Stream:** Alignment of raw reads back to the contigs using Bowtie2 to generate sorted alignment files (`.bam`) for coverage estimation.

Downstream execution relies strictly on the convergence of these outputs. **GraphBin2** requires three simultaneous inputs to execute label propagation: the assembly graph (`.gfa`), the formatted contig bins, and the contig coverage profiles. Subsequently, the **Read Back-Mapping** module requires both the refined contig labels generated by GraphBin2 and the read-to-contig alignment coordinates stored within Bowtie2's `.bam` files.

B. Assembly Graph Construction: MEGAHIT

The core requirement for topological label propagation is a structured assembly graph where nodes represent contigs and edges denote sequence overlaps. While traditional assemblers such as SPAdes preserve highly detailed, branch-rich scaffold connectivity, they incur prohibitive memory footprints (exceeding 80 GB Max RSS) and execution times on large metagenomic datasets.

To overcome this bottleneck, we deploy **MEGAHIT**, an assembler optimized for large-scale metagenomic datasets. It constructs de Bruijn Graphs by leveraging a compressed data structure based on the Burrows-Wheeler Transform. The assembly process operates through an iterative k-mer strategy, dynamically updating the compressed graph representation across increasing k-mer lengths (k_{min} to k_{max}). During graph traversal, MEGAHIT applies pruning to remove low-coverage dead-ends and linearize spurious bubbles caused by sequencing errors or minor strain variations. The resulting output generates a graph structure suitable for label propagation, while reducing the memory consumption, which is exactly what we want to achieve.

C. Initial Binning and "The Processor" Translation Module

Since GraphBin2 is designed to refine contiguous genomic fragments rather than isolated short sequences, the taxonomic binning from which the propagation starts must be performed at the contig level. We execute Kraken2 with a compressed reference database (Kraken2 Mini, 8 GB) on the assembled contigs to establish initial taxonomic seeds.

However, the output of Kraken2 and the initial binning required by Graphbin2 are incompatible. Indeed, Kraken2 outputs the given contigs mapped to arbitrary NCBI Taxonomy identifiers, with the possibility of retrieving unclassified nodes in case of uncertainty in the algorithm. In contrast, GraphBin2 requires the contigs to be mapped to sequential integers representing the bins and to have a strict node naming syntax that depends on the type of assembler graph used. To bridge this gap, we engineered a custom post-processing module. The module executes the following steps:

1) **Filtering and Quality Control:** strips unclassified contigs from the initial assignment file to prevent uninformative nodes from acting as topological sinks.
2) **Taxonomic Generalization:** traverses the NCBI taxonomy tree using a local nodes file to map lower-rank assignments (e.g., strain or subspecies level) to target standard ranks (species or genus), ensuring hierarchical consistency.
3) **Abundance Validation:** cross-references assigned contigs against the coverage alignment matrix, dropping zero-coverage anomalies.
4) **ID Re-mapping:** maps NCBI TaxIDs into a continuous sequence of positive integers and reformats contig headers to conform explicitly to MEGAHIT's node identifier syntax.

D. Abundance Estimation and Label Propagation

Graph-based label propagation cannot rely solely on topological connectivity; highly abundant contigs must be distinguished from low-coverage fragments to prevent misleading potential binning. To do so, we align the original raw FASTQ reads back onto the assembled contigs using **Bowtie2**. From the resulting sorted Binary Alignment Map (`.bam`) files, depth of coverage profiles are computed for each contig:

$$\text{Coverage}(C_i) = \frac{\sum_{j=1}^{M} L_{\text{read},j}}{L_{\text{contig},i}} \tag{1}$$

where M is the number of reads mapped to contig C_i, L_{read} is the read length, and $L_{\text{contig},i}$ is the sequence length of contig i.

The refined contig assignments, the assembly graph topology (`.gfa`), and the abundance vectors are passed to **GraphBin2**. The propagation algorithm uses a Min-Heap priority queue driven by contig coverage differences. Unclassified contigs at distance $d = 1$ from labeled seeds are iteratively extracted and evaluated. If the coverage of an unclassified node closely matches the mean coverage of an adjacent bin, the node adopts that bin's label. Furthermore, GraphBin2 evaluates a multi-labeling criteria: if a contig's coverage equals the sum of two adjacent distinct taxonomic bins ($\text{Coverage}_C \approx \text{Coverage}_A + \text{Coverage}_B$), it is duplicated across both bins, correctly capturing shared genomic elements.

E. Read Back-Mapping Module

The primary objective of our framework is read-level taxonomic classification to enable direct comparison with standard read classifiers. Since GraphBin2 outputs refined labels strictly for contigs, we developed a downstream **Read Back-Mapping** module.

The module parses the coordinates stored within the Bowtie2 `.bam` alignment files. For every read r_k, the algorithm retrieves its parent contig C_i and verifies the alignment integrity:

$$\text{Assign}(r_k) = \begin{cases} \text{Bin}(C_i), & \text{if MAPQ}(r_k) \geq Q_{thresh} \\ \text{Unclassified}, & \text{otherwise} \end{cases} \tag{2}$$

By setting a Mapping Quality threshold ($Q_{thresh} = 30$), we filter out ambiguous multi-mapping reads. The module seamlessly propagates the refined, graph-corrected contig bin back to all constituent raw reads, yielding the final read-level taxonomic classification output.

III. Experimental Setup & Benchmark

We define a rigorous benchmark where three taxonomic binning approaches of our interest are compared to each other:

1) **Kraken2 Standard**, based on the complete Kraken2 Standard reference database;
2) **Kraken2 Mini**, based on the reduced database of 8GB;
3) **Kraken2 Mini + GraphBin2**, where the taxonomic assignments generated by Kraken2 Mini are propagated through the assembly graph using GraphBin2, as described in the previous section.

We perform two types of evaluation at the species-level. First, we use the AMBER software package to assess (through some specific metrics) the taxonomic **binning quality** of the different strategies. Then, we evaluate the **sustainability** of the proposed pipeline by including Snakemake's built-in benchmark directive in each rule of the snakefile. In this way, we can store the exact runtime and peak memory consumption of each stage in a readable CSV file.

A. Dataset

We performed the evaluation on sample 0 of the **CAMI II Marine dataset**, a simulated marine metagenome composed of ten samples, for a total size of 50 Gb. Each sample consists of Illumina HiSeq paired-end short reads, all 150 bp long, and is associated with a Gold Standard, which reports the taxonomic origin (TaxID) of each read. This allows us to evaluate the predictions generated by the taxonomic binning methods against a known ground truth on a read-by-read level.

Sample 0 contains 33,294,790 reads, corresponding to 4,994,218,500 bp. Of these reads, approximately 90% belong to Bacteria, 8% to Archea, and 0.001% to Viruses. Only 259 species are effectively observed in the sample. A total of 28,394,156 reads (85.28%) are assigned to a species-level taxon, while the remaining 4,900,634 reads (14.72%) are either assigned at another taxonomic level or are left unassigned. The observed species are highly unevenly distributed, with abundances ranging approximately from 0.001% (Marivivens sp.) to 11.77% (Nitrosococcus halophilus). The number of reads per species ranges from 412 to 3,919,498, with a mean of 109,630 reads per species. This uneven species-abundance distribution provides a challenging setting for evaluating taxonomic binning methods.

B. Evaluation preprocessing

To evaluate the taxonomic binning quality through AMBER, we developed a script to convert the output of the different approaches and the Gold Standard into the standard CAMI `.binning` format required by AMBER. This format expects each read to be associated with a single row containing its sequence ID, its predicted/true bin and taxon, and, eventually, its length.

In particular,

- For Kraken2, each classified read is simply associated with its predicted TaxID.
- For GraphBin2, the output initially contains anonymous bin identifiers such as res.001. Therefore, each anonymous bin is translated back to the corresponding TaxID through the mapping file generated by the processor.
- For the Gold Standard, we convert the CAMI Gold Standard by adding the column LENGTH, which contains the length of each read. This step is required by AMBER to correctly compute the base-pair-weigthed metrics.

Furthermore, since the CAMI II Marine dataset contains paired-end reads, forward and reverse reads are treated as separate sequencing reads during the evaluation, consistently across the Gold Standard and all the different strategies. Finally, for

assessing taxonomic binning, AMBER also requires the NCBI taxonomy database adopted by the dataset to interpret the hierarchical relationships between taxa.

C. Performance metrics

AMBER computes several performance metrics by comparing the binnings predicted by the different methods with the Gold Standard assignment of the raw reads. Since the predicted bins do not have an intrinsic taxonomic label, AMBER maps each bin to the taxon that best represents it, i.e., the taxon that contributes the greatest number of base pairs (or sequences) to that bin. This mapping is then used to determine the true positives (TP), false positives (FP), and false negatives (FN), from which the following quality metrics are defined. The metrics can be divided into two main categories.

1) Metrics per bin: At the bin level, AMBER reports the **average purity** $\overline{p}$ and the **average completeness** $\overline{r}$, also known as macro-averaged precision and macro-averaged recall. The first is defined as the arithmetic mean of the purities of the predicted bins, while the second is defined as the arithmetic mean of their completenesses over all reference taxa (including those not recovered by any predicted bin). These are macro-averages, where all bins have the same weight, regardless of their size.

$$\overline{p} = \frac{1}{n_p} \sum_x p_x \qquad \text{where } n_p \text{ is the number of predicted bins.} \tag{3}$$

$$\overline{r} = \frac{1}{n_r} \sum_x r_x \qquad \text{where } n_r \text{ is the number of reference taxa.} \tag{4}$$

The purity p_x of a bin x measures the fraction of that bin that belongs to its mapped reference taxon, so it represents the specificity of the bin.

$$p_x = \frac{TP_x}{TP_x + FP_x} \tag{5}$$

Here, TP_x is the number of base pairs (or sequences) of the mapped reference taxon that are correctly assigned to x, while FP_x is the number of base pairs (or sequences) belonging to other taxa and incorrectly assigned to x.

The completeness r_x of a bin x measures the fraction of the mapped reference taxon recovered by the bin, so it represents the taxonomic recovery of the bin.

$$r_x = \frac{TP_x}{TP_x + FN_x} \tag{6}$$

Here, FN_x is the number of base pairs (or sequences) belonging to the mapped reference taxon that are assigned to other bins or left unclassified.

AMBER also reports the **F1 score**, which combines the previous two metrics through their harmonic mean.

$$\text{F1-score}_x = 2\frac{p_x r_x}{p_x + r_x} \tag{7}$$

2) Metrics per sample: Since some taxa are more abundant than others in most datasets, AMBER also reports other metrics that evaluate the performance of the binning at the sample level rather than the average quality of the individual bins.

The **accuracy** a is defined as the fraction of correctly assigned base pairs (or sequences) among all those in the dataset, so it measures both the quality of the predictions and the amount of data left unclassified.

$$a = \frac{\sum_x TP_x}{U + \sum_x (TP_x + FP_x)} \tag{8}$$

Here, U is the number of unclassified base pairs (or sequences).

The **sample purity** $\overline{p}_{bp}$ and the **sample completeness** $\overline{r}_{bp}$ take into account the size of the bins by assigning higher weights to larger bins, in contrast to the related metrics per bin described above.

$$\overline{p}_{bp} = \frac{\sum_x TP_x}{\sum_x (TP_x + FP_x)} \tag{9}$$

$$\overline{r}_{bp} = \frac{\sum_x TP_x}{\sum_x (TP_x + FN_x)} \tag{10}$$

The **Rand Index** (RI) indicates the similarity between the predicted and the Gold Standard partitioning of the assigned raw reads. The RI is computed by considering pairs of assigned sequences: pairs belonging to the same taxon and placed in the same predicted bin are considered true positives TP, while pairs of different taxa and placed in different bins are considered true negatives TN. Then, the RI is defined as the fraction of correctly classified pairs (TP or TN) among all possible pairs of reads. The **Adjusted Rand Index** (ARI) normalizes the RI by considering its expected value.

The **percentage of binned sequences** (or base pairs) measures the fraction of raw reads that have been assigned to some predicted bin.

IV. Results

A. Taxonomic binning performance

Since all raw reads have identical lengths (150 bp) in the CAMI II Marine dataset, corresponding base pair-based and sequence-based metrics from AMBER yield equal values. For this reason, only one value is reported for each metric in Table I, which shows the evaluation results at the species level.

The results reveal that Kraken2 Standard remains the strongest overall method at the species level. It classifies approximately 84.4% of raw reads, achieving at the sample level a purity of 0.953, a completeness of 0.804, and an F1 score of 0.872.

As expected, Kraken2 Mini leads to a loss of taxonomic recovery, assigning only 35.1% of the reads and decreasing the sample completeness to 0.341 and the F1 score to 0.505. However, the sample purity is still high (0.972), showing that the main drawback of Kraken2 Mini is not the low accuracy

TABLE I
AMBER EVALUATION RESULTS ON SAMPLE 0 OF THE CAMI II MARINE DATASET AT THE SPECIES LEVEL. THE REPORTED VALUES ARE VALID FOR BOTH BP-BASED AND SEQ-BASED METRICS.

Metric	**Gold Standard**	**Kraken2 Standard**	**Kraken2 Mini**	**Kraken2 Mini + GraphBin2**
Quality of bins: all bins have the same weight				
Average purity	1.000	0.038	0.043	0.274
Average completeness	1.000	0.782	0.327	0.267
F1 score	1.000	0.072	0.076	0.270
Quality for sample				
Accuracy	1.000	0.804	0.341	0.673
Misclassification rate	0.000	0.047	0.028	0.074
Purity	1.000	0.953	0.972	0.926
Completeness	1.000	0.804	0.341	0.673
F1 score	1.000	0.872	0.505	0.779
Rand index	1.000	0.999	1.000	0.999
Adjusted Rand index	1.000	0.994	0.996	0.989
Percentage of binned sequences	1.000	0.844	0.351	0.726

TABLE II
COMPUTATIONAL REQUIREMENTS OF THE PIPELINE COMPONENTS ON THE DEI'S CLUSTER PLATFORM.

Tool	**Wall-clock time**	**Max RSS (GB)**
Kraken2 Standard	21 min 29 s	102.9
Kraken2 Mini	2 min 29 s	9.4
MEGAHIT	2 h 50 min 37 s	6.5
Bowtie2	1 h 19 min 26 s	13.8
GraphBin2	4 h 59 min 20 s	2.2

of its predictions, but the limited amount of raw reads it can classify due to missing reference k-mers in the reduced index.

Starting from Kraken2 Mini's output, GraphBin2 improves these results, increasing the percentage of binned reads to 72.6%, the sample completeness to 0.673, and, consequently, the F1 score to 0.779. So, GraphBin2 rescues a great fraction of the reads that Kraken2 Mini leaves unclassified, while preserving high consistency. Moreover, it provides a high sample purity (0.926), a low misclassification rate (0.074) and a high Adjusted Rand index (0.989), indicating a strong agreement between the resulting partitioning of the reads and the Gold Standard.

This trend shown by the metrics per sample is similar for the metrics per bin. The only relevant difference regards our hybrid approach, which provides at the same time the highest average purity (0.274) and the lowest average completeness (0.267). Therefore, at the species level, our framework makes more consistent predictions when creating a bin, but recovers much less information about each taxon compared to Kraken2 Standard, which reaches an average completeness of 0.782.

B. Computational requirements

Table II shows the exact execution time and maximum memory (RAM) consumption of the different tools used in our framework.

As we can see, Kraken2 Standard requires approximately 103 GB of RAM to store the related taxonomic index, thus becoming unfeasible for conventional commodity hardware although its execution time is relatively low. Kraken2 Mini reduces these computational requirements, running in less than three minutes and requiring only 9.4 GB of RAM.

GraphBin2 has a low memory footprint, requiring only 2.2 GB of RAM to process the assembly graph, but it executes for approximately 5 hours, much longer than the Kraken2 stages. The assembly and read back-mapping steps are also temporally heavy, requiring approximately four hours for their execution, while preserving a peak memory usage under 14 GB of RAM.

V. DISCUSSION

The results show that reducing the size of a taxonomic classifier database introduces a great loss of taxonomic coverage at the species level. In particular, Kraken2 Mini drops the percentage of binned reads from 84.4% of Kraken2 Standard to 35.1%. However, the sample purity does not change significantly, indicating that using an incomplete database does not necessarily lead to incorrect assignments, but to leave many reads unclassified.

In the proposed workflow, GraphBin2 exploits the assembly graph topology to propagate labels to unclassified reads, thus increasing the percentage of binned reads to 72.6%. The sample completeness also improves to 0.673, getting closer to the threshold achieved by the Kraken2 Standard classifier (0.804) and clearly limiting the gap introduced by the memory-constrained database. This approach also preserves high sam-

ple purity (0.926), high Adjusted Rand index (0.989), and low misclassification rate (0.074), showing that GraphBin2 ensures the correctness of the predictions, as well as an increase in the number of binned reads.

However, GraphBin2 does not achieve the performance of Kraken2 Standard, which remains the best in terms of binned reads, sample completeness and F1 score at the species level. Therefore, the proposed framework should not be seen as a replacement for a standard taxonomic classifier, but rather as a resource-efficient workflow to rescue a large amount of "lost" information from a lightweight classifier.

This trade-off is also confirmed by the computational requirements of our pipeline. GraphBin2 introduces a relevant additional runtime to the Kraken2 classification stage while ensuring a relatively small memory footprint. Moreover, in our pipeline the assembly and read back-mapping stages also represent expensive stages from a temporal point of view. We conclude that the proposed approach represents an interesting solution in memory-constrained scenarios where the assembly and read mapping phases are already performed for downstream metagenomic analysis.

VI. Conclusions

The evaluation on the CAMI II Marine dataset shows a clear trade-off between database size, taxonomic recovery, accuracy, and computational requirements. A reduced Kraken2 database decreases the memory consumption of a complete reference database, but also increases the number of unbinned sequences. In the specific case, the Kraken2 Mini's classifier assigns 35.1% of the reads at the species level, very far from 84.4% achieved by Kraken2 Standard.

The proposed approach integrates a lightweight classifier with GraphBin2, which processes the assembly graph to extend the taxonomic information to many unclassified sequences. Starting from Kraken2 Mini's predictions, GraphBin2 increases the fraction of binned reads to 72.6% and the sample completeness to 0.673 at the species level, without any expense of precision and especially ensuring low memory usage.

Therefore, the results confirm our initial hypothesis: the assembly-graph topology can be exploited to rescue a substantial fraction of the taxonomic information that is not available while using a reduced taxonomic database. Even if the proposed framework does not outperform Kraken2 Standard in overall recovery, completeness and execution time, it significantly decreases the performance gap, while preserving high purity and avoiding the huge memory footprint due to the standard database. Future works could focus on optimizing GraphBin2 to reduce its execution time, improving the filtering of the recovered assignments, and investigating alternative assemblers and graph-based classification strategies.

References

[1] Vijini Mallawaarachchi, Anuradha Wickramarachchi, Yu Lin, *GraphBin: Refined binning of metagenomic contigs using assembly graphs*, Bioinformatics, Volume 36, Issue 11, June 2020, Pages 3307–3313

[2] Fernando Meyer, Peter Hofmann, Peter Belmann, Ruben Garrido-Oter, Adrian Fritz, Alexander Sczyrba, Alice C McHardy, *AMBER: Assessment of Metagenome BinnERs*, GigaScience, Volume 7, Issue 6, June 2018

Appendix

A. Environment and Repository Layout

The complete pipeline is orchestrated via Snakemake, with source code, configuration files, and custom Python scripts hosted at: https://github.com/Pizz0x/TaxonomicClassification.

Dependencies are isolated via rule-level Conda environment definitions located in the `envs/` directory (`assembler.yml`, `graphbin2.yml` and `amber.yml`). External input paths (e.g., CAMI FASTQ reads, Kraken2 databases, and NCBI taxonomy files) and execution parameters are centralized in `config.yaml`. Custom conversion and processing scripts are located in the `scripts/` directory. Snakemake handles environment creation automatically when invoked with `--use-conda`.

B. Pipeline Execution Flow and Parameters

The end-to-end workflow integrates tool dependencies and custom scripts across four sequential operational phases:

a) Phase 1: Assembly and Graph Preprocessing: Raw paired-end reads from the CAMI dataset are provided as a single interleaved FASTQ file (`cami_reads`) and assembled into contigs using `megahit` via the `--12` flag (for interleaved paired-end input) using 32 threads. The resulting de Bruijn graph is converted to FASTG format via `megahit_toolkit contig2fastg` and transformed into GFA topology using `scripts/fastg2gfa.py`. Alternatively, `metaspades.py` can be selected.

b) Phase 2: Taxonomic Seeding and Abundance Profiling: Contigs are classified with `kraken2` using the Kraken2 Mini database (8 GB, `--threads 32`). Baseline read-level classifications are generated on raw reads for both Mini and Standard (100 GB) databases (`--fastq-input --interleaved`). For coverage estimation, raw reads are aligned back to contigs via `bowtie2` (`-p 32 --interleaved`), sorted with `samtools sort -@ 32`, and converted to abundance profiles via `samtools idxstats`.

c) Phase 3: Label Translation and Graph Propagation: The module `scripts/processor.py` parses contig assignments (`--rank species --assembler megahit`), filters unclassified nodes, traverses the NCBI taxonomy tree (`nodes.dmp`), and maps TaxIDs to sequential integers in `bin_taxid_mapping.csv`. Next, `graphbin2` processes the assembly graph (`.gfa`), abundance profiles, and contig bins to propagate labels across unclassified contigs using a coverage-weighted priority queue.

d) Phase 4: Read Back-Mapping and Evaluation: To assign labels back to raw reads, `scripts/reads_binning.py` cross-references GraphBin2 contig labels with alignment coordinates in `alignment.bam`, retaining primary alignments with $MAPQ \geq 30$. Finally,

`scripts/generate_binning.py` formats the classifications (`--length 150`), and `amber.py` computes purity, completeness, F1 score, and ARI against the CAMI gold standard.

C. Prerequisites and Command-Line Execution

Before executing the workflow, input dataset paths, reference database directories, and NCBI taxonomy locations must be configured in `config.yaml`.

Due to the high memory requirement of the Kraken2 Standard index (~ 103 GB RAM), running the full workflow requires a high-memory environment to execute the pipeline directly on a dedicated high-memory server:

```
snakemake --use-conda --cores 32
```

For HPC execution via Slurm, the workflow is submitted using the `sbatch` command with the provided batch script `run_pipeline.sh`, which is available on `/nfsd/bcb/bcbg/demo_pizzo/project`.